Customer Churn Prediction Model

Software Engineering Mini Project: Deliverable 1

Machine learning classification using XGBoost

This deliverable has three parts:

- Part A: Software Requirements Specification (SRS)
- Part B: Software Architecture and Design Specification (SAD)
- Part C: Software Test Plan and Test Cases

Detail	**Value**
Course	Software Engineering Mini Project
Project title	Customer Churn Prediction Model
Institution	PES University
Team name	Hackforge
Team members (SRN)	1. Akshaya B (PES1UG24AM026) 2. Ashwin C H (PES1UG24AM900) 3. Amar Patil (PES1UG24AM031)
Instructor	Cvsn Reddy
Submission date	12 October 2026

Contents

Part A: Software Requirements Specification (SRS)

1. Introduction
2. Overall Description
3. Specific Requirements
4. Supporting Information

Part B: Software Architecture and Design Specification (SAD)

1. Introduction
2. Architectural Design
3. Detailed Design

Part C: Software Test Plan and Test Cases

1. Introduction
2. Test Items
3. Features to be Tested
4. Features Not to be Tested
5. Approach
6. Item Pass/Fail Criteria
7. Entry, Exit, Suspension and Resumption Criteria
8. Test Deliverables
9. Responsibilities and Schedule
10. Risks and Contingencies
11. Test Cases
12. Traceability Matrix

Appendix A: Experiment Record Template

Part A: Software Requirements Specification (SRS)

We followed the general layout of IEEE 830 for this part. Our project has two parts: a command line Python script that trains the model, and a Streamlit web app that uses the saved model to make predictions and to check how well the model does. The requirements below cover both. Things we might add later are listed separately so they are not confused with what is already built.

1. Introduction

1.1 Purpose

This SRS explains what our Customer Churn Prediction Model should do, what limits it has, and what quality we expect from it. It is written for our team, our instructor and anyone reviewing the project. The training script learns from customer data in a CSV file, and the web app lets a user enter one customer's details and see how likely that customer is to churn (leave the service). Part B explains how the program is built inside, and Part C explains how we test it.

1.2 Scope

The project is a Python machine learning pipeline built on the Telco Customer Churn dataset. The main script loads the CSV, prepares the features, trains an XGBoost classifier, tries out different hyperparameter combinations using GridSearchCV with stratified cross validation, checks how well the model does, prints the results and saves the trained model and encoders so they can be used again.

A Streamlit web app (app.py) is built on top of this. It loads the saved model and encoders, lets a user type in a customer profile or pick a sample one, checks the input, and shows the churn probability, a churn / stay result and a risk tier. A second page shows the model's results on the held-out test set and can re-run that evaluation. The app never retrains the model.

This is an academic project. The aim is to show how churn classification works and how we judge a model using recall, precision and accuracy. It is not a real customer management system. It does not contact customers, it is not connected to any live telecom system, and its predictions do not guarantee that a customer will actually leave.

1.3 Definitions and abbreviations

Term	Meaning
Customer churn	When a customer leaves or stops using the service. In our dataset this is the Churn column.
XGBoost	A library that builds many small decision trees one after another, each fixing the mistakes of the last. We use it as our final model.
GridSearchCV	A scikit-learn tool that tries every combination of the hyperparameters we give it and picks the best one using cross validation.
Stratified K-Fold	A way of splitting data into folds so that each fold has roughly the same share of churn and non churn customers.
Precision	Out of the customers we predicted would churn, how many actually churned.
Recall	Out of all the customers who actually churned, how many we caught.
Accuracy	Out of all predictions, how many were correct.
Class imbalance	When one class (here, non churn) has many more rows than the other.
Encoders	Saved rules that turn text columns (like contract type) into numbers the model can use.
False positive	We predicted churn, but the customer did not churn.
False negative	The customer churned, but we predicted they would stay.
Streamlit	A Python library for building simple web apps. We use it for our prediction and evaluation pages.
Decision threshold	The churn probability at or above which a customer is labelled as churn. The default is 0.50 and the user can change it in the app.
Risk tier	A simple label for the churn probability: Low (below 35%), Moderate (35% to 65%) or High (above 65%). These cut-offs are business rules we chose, not tuned values.
Preset	A saved example customer profile that fills in the whole form in one click.
Specificity	Out of all the customers who stayed, how many we correctly predicted would stay.
ROC-AUC	How well the churn probability ranks churners above customers who stay (0.5 means no better than guessing, 1.0 is perfect).

1.4 Overview

Section 2 covers the product, its users, constraints and dependencies. Section 3 lists the interfaces, use cases, functional and non functional requirements and security requirements. Section 4 describes the dataset and how to run the training script and the web app. Part B covers the design and Part C covers testing.

2. Overall Description

2.1 Product perspective

The product has two parts, and both run on a local machine. The training script (01_churn_prediction_xgboost.py) is run from the terminal. It reads the Telco churn CSV, processes it, trains and evaluates the model, and saves the model and encoder files. The Streamlit web app (app.py) is started with streamlit run app.py and opens in the browser. It loads the saved files through our prediction_pipeline.py module, so it can make predictions without retraining. Nothing is hosted online. Figure A1 shows the training side; the web app side is shown in Part B, Figure B4.

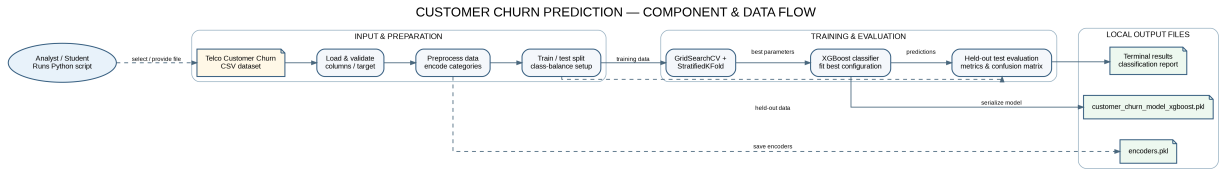


Figure A1. Components and data flow of the project.

2.2 Product functions

- Read the Telco Customer Churn CSV file.
- Separate the target column (Churn) from the input columns.
- Convert text columns into numbers using our encoders.
- Split the data into training and test sets in a repeatable way.
- Use class weighting to deal with class imbalance instead of throwing away rows.
- Try the XGBoost hyperparameter grid using GridSearchCV and stratified cross validation.
- Evaluate the chosen model with accuracy, precision, recall, a confusion matrix and a classification report.

- Save the trained model and the encoders as files for later use.
- Let a user enter one customer's details in a web form, or load one of four sample presets.
- Check the form input before predicting (missing fields, allowed values, number ranges and service combinations) and list every problem found.
- Show the churn probability, the churn / stay result at an adjustable decision threshold, a risk tier (Low, Moderate or High) and a suggested action.
- Show the saved test-set results and re-run the evaluation on the held-out test set from the app, with a confusion matrix chart.

2.3 User characteristics

The expected user is a student, instructor or reviewer who can run a Python script, install packages and understand basic classification metrics. They do not need to be ML experts. For the web app, the user (for example someone in a customer success team) only needs a browser and a basic idea of what the customer fields mean. No coding is needed to make a prediction.

2.4 Constraints

- The program depends on the dataset having the columns and target that the script expects.
- How well the model does is limited by the quality and size of the dataset.
- Because of class imbalance, accuracy alone can be misleading, so we also look at recall and precision.
- This version runs only on a local machine. The web app is meant for demos and decision support, not as a hosted product. A REST API, CRM connection, user accounts or automatic retention campaigns are not included.
- Our results are from our own experiments and may not hold for future customers.
- The saved model must always be used with the encoders saved in the same run.

2.5 Assumptions and dependencies

- The CSV file is in the location the script expects and has the right columns.
- Python and all required libraries are installed.
- The Churn column contains the expected Yes / No values.
- There is enough data in the training and test sets to calculate meaningful metrics.
- The .pkl files are loaded with the same (or compatible) Python and library versions used to create them.

- The model and encoder files are in the project folder when the web app starts.
- Streamlit 1.36 or newer is installed (it is listed in requirements.txt).

2.6 Out of scope

- Connecting to a real telecom company's customer database.
- Sending emails, making calls, giving discounts or any other retention action.
- Cloud deployment, user accounts or a public, customer facing website. Our Streamlit app is a local tool for the team and for the demo.
- Guaranteeing that a customer will churn, or explaining exactly why they did.
- Production level monitoring, drift alerts or scheduled retraining.

3. Specific Requirements

3.1 External interface requirements

ID	Requirement	Priority	Verified by
UI-01	The terminal output should clearly label each metric and show that churn is the positive class.	Must	TC-06
UI-02	The web app shall show a form with a clear label and help text for each of the 19 customer fields, grouped into demographics, services and contract / billing.	Must	TC-14
UI-03	Input problems shall be shown together in one clear error message, and no prediction shall be shown while the input is invalid.	Must	TC-16
UI-04	The result shall show the churn probability, the churn / stay label, the risk tier and a note that it is an estimate, not a certainty.	Must	TC-17
UI-05	The evaluation page shall show the test-set metrics and a confusion matrix chart.	Must	TC-19
SW-01	The script should load the CSV and show a clear error if the file is missing or cannot be read.	Must	TC-01
SW-02	After a successful run, the model and encoders should be saved to the expected file paths.	Must	TC-08
SW-03	The web app shall load the saved model and encoders once at start-up and show a clear error if either file is missing.	Must	TC-20

HW-01	The program should run on a normal laptop or desktop. A GPU is not needed. The web app needs a modern web browser.	Should	TC-10
COM-01	All data is processed locally. The web app runs on the user's own machine (localhost) and no customer data is sent to any outside service.	Must	TC-11

3.2 Use case view

For the training script, the main actor is the student or analyst. They run the script, look at the results and check that the output files were saved. For the web app, the actor is a customer success user (or a student during the demo) who enters a customer's details, reads the prediction and checks the model's test results. The system boundary is our Python code and the local files. There is no live telecom system involved.

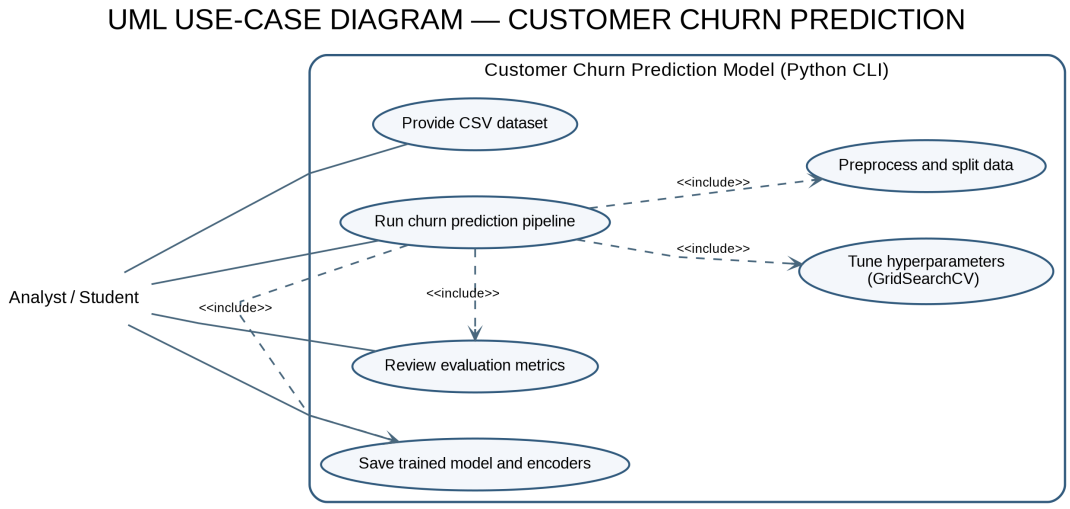


Figure A2. Use case diagram for the training script (command line).

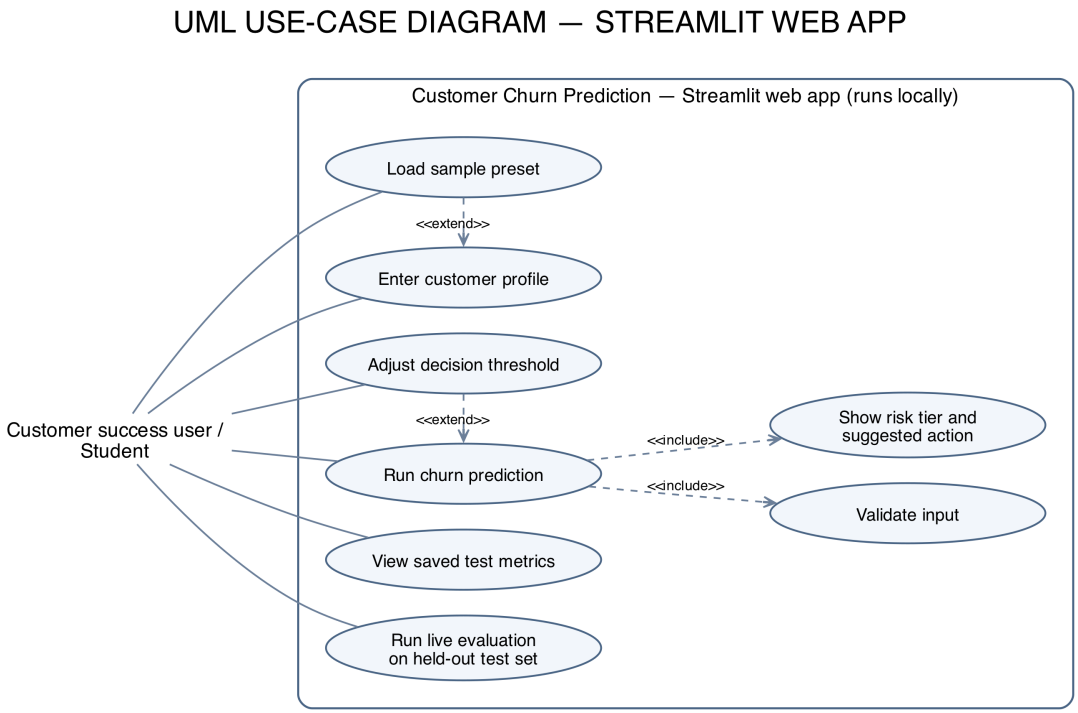


Figure A3. Use case diagram for the Streamlit web app.

Use case	**Description**	**Related requirements**
UC-01: Load dataset	Read the Telco churn CSV and make sure the data is available.	FR-01, FR-02
UC-02: Prepare features	Separate the target, encode the text columns and get the inputs ready.	FR-03, FR-04
UC-03: Train model	Train XGBoost and run the hyperparameter search.	FR-05, FR-06
UC-04: Evaluate model	Print the metrics and confusion matrix on the test data.	FR-07, FR-08
UC-05: Save files	Save the trained model and the encoders.	FR-09
UC-06: Handle bad input	Stop and show a clear error if the input is missing or wrong.	FR-02, NFR-02
UC-07: Enter customer profile	Fill in the web form by hand, or load a sample preset and edit it.	FR-15, FR-16
UC-08: Validate input	Check the form values and list every problem before any prediction is made.	FR-17

UC-09: Predict churn	Show the churn probability, the label at the chosen threshold, the risk tier and a suggested action.	FR-18, FR-19
UC-10: Check model performance	View the saved test-set results or re-run the evaluation on the held-out test set.	FR-20, FR-21

3.3 Functional requirements

Must means the requirement has to be met for the project to be accepted. Should means it is good to have.

ID	Requirement	Priority	Verified by
FR-01	The system shall load the Telco Customer Churn CSV from the set path.	Must	TC-01
FR-02	If the dataset is missing or unreadable, the system shall show a clear error and not continue with empty data.	Must	TC-01, TC-02
FR-03	The system shall separate the Churn target from the input columns before training.	Must	TC-03
FR-04	The system shall convert text columns into numbers using our encoders.	Must	TC-03
FR-05	The system shall train an XGBoost classifier on the training data.	Must	TC-04
FR-06	The system shall use GridSearchCV with stratified cross validation to compare hyperparameter combinations.	Must	TC-05

FR-07	The system shall calculate and print accuracy, precision, recall and a classification report.	Must	TC-06
FR-08	The system shall show a confusion matrix so false positives and false negatives can be seen.	Must	TC-06
FR-09	After a successful run, the system shall save the trained model and encoders.	Must	TC-08
FR-10	The system shall use class weighting during training without changing the test set.	Must	TC-04, TC-07
FR-11	The system shall show the best hyperparameters found by the search.	Should	TC-05
FR-12	The output shall make it clear which part is training setup and which part is evaluation results.	Must	TC-06
FR-13	The project documentation shall explain the dataset, the model, the metrics and the limitations.	Must	TC-12
FR-14	The saved model and encoders shall be loadable later for making predictions.	Should	TC-09
FR-15	The web app shall let the user enter all 19 customer fields used by the model.	Must	TC-14
FR-16	The web app shall offer sample customer presets (high risk, borderline, low risk and phone-only) that fill the form, and a way to clear the form.	Should	TC-15

FR-17	Before predicting, the system shall check that every field is filled in, categories are allowed values, numbers are valid and in range, and the billing and service values make sense together, and shall list every problem found.	Must	TC-16
FR-18	The system shall show the churn probability and the churn / stay label for the entered customer, using the saved model and encoders.	Must	TC-17
FR-19	The user shall be able to change the decision threshold (default 0.50). The system shall also show a risk tier (Low below 35%, Moderate 35% to 65%, High above 65%) with a suggested action.	Should	TC-18
FR-20	The evaluation page shall show accuracy, precision, recall, F1-score, specificity, ROC-AUC and the confusion matrix for the held-out test set.	Must	TC-19
FR-21	The evaluation page shall be able to re-score the saved model on the held-out test set and say whether the results match the saved benchmark.	Should	TC-19

3.4 Non functional requirements

ID	Category	Requirement	Verified by
NFR-01	Performance	We will note down how long a normal training run takes on our machine instead of guessing it.	TC-10
NFR-02	Reliability	If the dataset or required columns are missing, the run should fail with a clear error and not print fake metrics.	TC-01, TC-02
NFR-03	Reproducibility	With the same data, random seed and library versions, running again should give the same split and similar results.	TC-13
NFR-04	Model quality	We report recall and precision along with accuracy, because missing a churner and raising a false alarm cost different things.	TC-06
NFR-05	Maintainability	Loading, preprocessing, training, evaluation and saving should be clearly separate steps in the script. Validation, prediction and evaluation code should live in prediction_pipeline.py, separate from the web page layout in app.py.	TC-12
NFR-06	Portability	Someone should be able to install the dependencies in a fresh environment and run the script without our local venv folder.	TC-10
NFR-07	Integrity	The model file and encoder file should always come from the same training run.	TC-09

NFR-08	Usability	Metric names and error messages should be easy to understand for a student with basic ML knowledge.	TC-06, TC-02
NFR-09	Privacy	The model does not need names, phone numbers or other personal identifiers. Any such columns are dropped from the inputs.	TC-11
NFR-10	Resource use	If a library or file is missing, the script should stop with a visible error instead of looking like it finished.	TC-01, TC-10
NFR-11	Responsiveness	Once the app has loaded, a single prediction should appear within about 2 seconds on a normal laptop.	TC-17
NFR-12	Testability	The pipeline and the web app should have automated tests (pytest) that run on every push and pull request through GitHub Actions.	TC-21

3.5 Performance results

In our runs the model got around 76% accuracy, 75% recall and 53% precision for the churn class. We care most about recall because it tells us how many real churners we actually catch. Precision is also important because it tells us how many of the flagged customers really churned. Since the classes are imbalanced, accuracy alone does not tell the full story.

We also brought the gap between training and test accuracy down from about 19 percentage points to about 7 after tuning the model and adding regularisation. These numbers are from our own runs and could be different on other data.

The web app's evaluation page re-scores the saved model on the same 1,409-customer test set and gets the same numbers. It also reports a specificity of about 76% and a ROC-AUC of about 0.84.

3.6 Design constraints

- XGBoost is the model used in this version.
- The same target and preprocessing must be used during training and prediction.
- The test set is kept separate and is not used for hyperparameter tuning.
- New data must be encoded with the saved encoders before using the saved model.
- Predictions are estimates based on past data, not certain outcomes.
- The web app must use the same saved model, encoders and column order as training. It never retrains the model.

3.7 Software quality attributes

The system should reliably finish the training run, give repeatable results so we can compare experiments, be easy to change when we tune the model, and be simple enough to demo in class. Since the results depend on the data and how we validate, we mention the limitations along with the metrics.

3.8 Security requirements

ID	Objective	Requirement	Verified by
SO-01	Confidentiality	Do not collect or store personal customer identifiers unless really needed.	TC-11
SO-02	Integrity	Keep the model and encoder files together and do not mix files from different runs.	TC-09
SO-03	Safe execution	Treat the CSV only as data. Nothing in it is ever run as code.	TC-02
SR-01	Data minimisation	Drop personal identifiers and unused columns from the model inputs.	TC-11
SR-02	Repository hygiene	Do not push passwords, private data or the local venv folder to a public repo.	TC-11
SR-03	File integrity	Write down the model and encoder file names and regenerate both together when the pipeline changes.	TC-09
SR-04	Local processing	The training pipeline does not send data or predictions to any outside service.	TC-11
SR-05	Input validation	Missing files, a missing target column or wrong data formats should give a controlled error. In the web app, missing or bad values are rejected with a readable message before they reach the model.	TC-01, TC-02
SR-06	Web app privacy	The web app runs on localhost only, is not deployed publicly, and does not save the customer details typed into the form.	TC-11

4. Supporting Information

4.1 Dataset details

Item	Description
Dataset	WA_Fn-UseC_-Telco-Customer-Churn.csv
Domain	Telecom customer churn
Target	Churn column (Yes / No)
Inputs	Customer details, services subscribed, contract type and billing information from the CSV.
Preprocessing	Encoding of text columns and other steps done in the Python script.
Class balance	Churn is the smaller class, so we use class weighting.
Limitation	Results may not carry over to a different company, time period or customer base.

4.2 How to run

1. Activate the Python environment (or install the dependencies in a new one).
2. Make sure the dataset file is in the expected location.
3. Run the script: python 01_churn_prediction_xgboost.py
4. Check the output: best parameters, accuracy, precision, recall, confusion matrix and classification report.
5. Check that customer_churn_model_xgboost.pkl and encoders.pkl were created or updated together.
6. Start the web app: streamlit run app.py, then open http://localhost:8501 in a browser.
7. Run the automated tests: pytest -v (this runs test_pipeline.py and test_app.py).

4.3 Implementation notes

- The repository has the training script, a reusable prediction_pipeline.py module, the Streamlit web app (app.py) with sample presets (customer_presets.py), and automated tests (test_pipeline.py and test_app.py) that run in GitHub Actions. There is no network API.

- Our grid search tried 12 hyperparameter combinations, each checked on 3 folds (36 fits in total).
- The local venv folder is just for our convenience and is not needed for a fresh install.
- All metrics in the final report are taken from the terminal output of an actual run.

Part B: Software Architecture and Design Specification (SAD)

This part follows the general layout of IEEE 1016. It describes how our local Python ML pipeline and the Streamlit web app on top of it are put together. A network API or cloud deployment is not part of the current design.

1. Introduction

1.1 Purpose and scope

This section explains how the program is organised, how the data moves through training and evaluation, how the output files are saved, how the web app uses them to make predictions, and how we deal with errors and privacy. It also links the requirements in Part A to the components here and the tests in Part C.

1.2 Design goals

- Keep it simple enough to run on a laptop: the training script from a terminal and the web app in a browser.
- Keep data preparation, training, evaluation and saving as separate steps.
- Handle class imbalance without throwing away training data.
- Use cross validation to pick model settings, without touching the test set.
- Make sure the saved model and encoders always match.
- Show clear results and do not overclaim what a prediction means.
- Keep the web page code separate from the validation, prediction and evaluation logic, so the logic can be tested without a browser.

2. Architectural Design

2.1 Architecture pattern

The program uses a simple pipeline style. One Python script runs each stage in order: load the data, preprocess, split into train and test, run the hyperparameter search, evaluate, and save the files. XGBoost is the model, and scikit-learn handles cross validation and the metrics.

The web app adds a simple layered style on top. app.py is the presentation layer: it draws the pages, the form and the results. prediction_pipeline.py and customer_presets.py are the logic layer: loading the saved files, cleaning and validating input, preprocessing, predicting and

evaluating. The .pkl files and the CSV are the data layer. The app only reads the files the training script saved.

Pattern / element	**Where it is used**	**Why**
Pipeline style	Stages inside the main script	Easy to follow the order of steps and easier to debug.
Estimator abstraction	XGBoost classifier	Keeps the model separate from data preparation and evaluation.
Cross validation search	GridSearchCV with StratifiedKFold	Compares parameter combinations while keeping the churn ratio similar in every fold.
Local file storage	Pickle files for model and encoders	Lets us reuse the model without retraining every time.
Layered (presentation / logic / data)	app.py, prediction_pipeline.py, saved .pkl files	Changes to the page layout do not touch the model logic, and the logic can be unit tested.
Caching	st.cache_resource and st.cache_data in app.py	The model is loaded once and the live evaluation is not recomputed on every click.

2.2 Components

Component	What it does	Input / output
Dataset loader	Reads the CSV file.	In: CSV path Out: DataFrame or a read error
Preprocessing / encoder	Separates the target and encodes text columns.	In: raw columns Out: model ready features and encoders
Split and class balance	Splits train and test data and sets up class weighting.	In: features and target Out: train/test sets and weight setting
Hyperparameter search	Runs GridSearchCV over XGBoost settings with stratified folds.	In: training data and parameter grid Out: best model and parameters
Evaluation	Predicts on the test set and calculates the metrics.	In: trained model and test data Out: report and confusion matrix
File writer	Saves the trained model and encoders.	In: model and encoders Out: .pkl files
Terminal output	Shows progress, chosen settings and metrics.	In: settings, metrics, errors Out: readable console messages
Prediction pipeline module (prediction_pipeline.py)	Loads the saved model and encoders, cleans and validates input, preprocesses it in the model's column order, predicts with a threshold and risk tier, and runs the evaluation.	In: customer record or dataset Out: prediction and risk tier, error list, or metrics
Web app (app.py)	Streamlit pages for prediction, model evaluation and project information.	In: form input and button clicks Out: results, charts and error messages in the browser
Customer presets (customer_presets.py)	Holds four example customer profiles for the form.	In: preset name Out: field values for the form
Automated tests and CI	test_pipeline.py and test_app.py check the pipeline, validation rules, presets and app pages. GitHub Actions runs them on every push and pull request.	In: code and saved files Out: pass / fail report

Figure B4 shows how the web app is layered on top of the files saved by the training script.

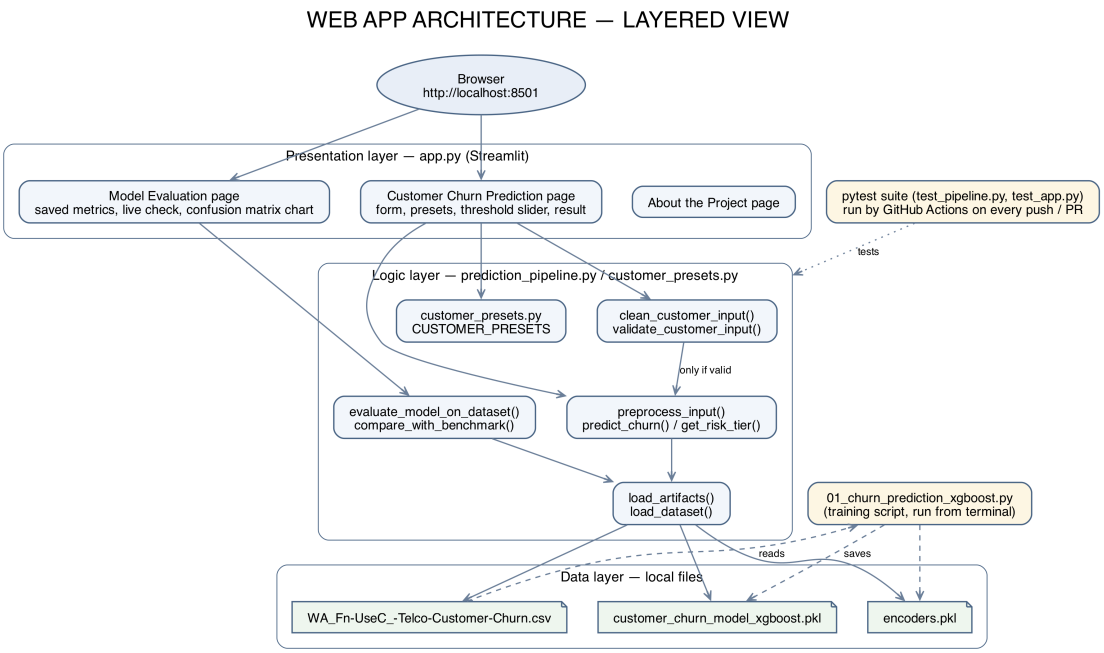


Figure B4. Layered view of the web app.

2.3 Technology stack

Technology	**Used for**
Python	Main language
XGBoost	Gradient boosted tree classifier
scikit-learn	Train/test split, StratifiedKFold, GridSearchCV and metrics
pandas / NumPy	Handling the table data and numeric work
pickle	Saving the model and encoders
CSV	Input dataset format
Streamlit	Web app pages, widgets and caching
Altair	Confusion matrix chart on the evaluation page
pytest	Automated tests, including Streamlit's AppTest for the app pages
GitHub Actions	Runs the test suite on every push and pull request to main

Exact library versions are listed in the requirements file in our repository.

2.4 Deployment view

The program runs on a student laptop or desktop. The user activates the Python environment and runs the training script in the terminal. Both the input data and the output files are stored locally. No GPU is needed.

The web app runs on the same machine. The user runs streamlit run app.py, which starts a local server, and opens http://localhost:8501 in the browser. The app reads the .pkl files and the CSV from the project folder. GitHub Actions runs the tests on GitHub's servers for every push and pull request, but the app itself is not deployed online.

2.5 Requirements to design mapping

Requirement IDs	Design component	How it is handled
FR-01, FR-02	Dataset loader	Reads the CSV and handles missing or unreadable files.
FR-03, FR-04	Preprocessing / encoder	Separates the target and encodes text columns.
FR-05, FR-06	XGBoost / hyperparameter search	Trains XGBoost and searches the parameter grid.
FR-07, FR-08, FR-12	Evaluation / terminal output	Prints the report, confusion matrix and labelled metrics.
FR-09, FR-14, NFR-07	File writer	Saves the model and its matching encoders.
FR-10, NFR-04	Training setup / evaluation	Uses class weighting and reports precision and recall.
NFR-02, NFR-10	Dataset loader / error handling	Stops with an error if something needed is missing.
NFR-03	Split and search setup	Uses fixed random seeds and stratified cross validation.
SR-01, SR-02, SR-04	Data preparation / repo practice	Removes identifiers and keeps private data and secrets out of GitHub.
COM-01, SR-04	Local CLI and local web app	The app only runs on localhost. All processing is local.
FR-15, FR-16, UI-02	Web app form / customer presets	Labelled form fields; presets and form values are kept in session state.
FR-17, SR-05, UI-03	Validation (clean_customer_input, validate_customer_input)	Checks every rule and returns a list of readable errors before predicting.
FR-18, FR-19, UI-04	Prediction (preprocess_input, predict_churn, get_risk_tier)	Same encoders and column order as training; threshold and risk tier applied to the probability.
FR-20, FR-21, UI-05	Evaluation (evaluate_model_on_dataset, compare_with_benchmark)	Re-scores the saved model on the held-out split and compares it with the saved results.
SW-03	Web app start-up	Loads the files once and shows an error if either is missing.
NFR-12	pytest suite and GitHub Actions	Runs all tests automatically.

2.6 Security design

There are three areas we need to think about: the input dataset, the running Python process, and the output files along with our public GitHub repo. We treat the dataset as untrusted input. The model and encoder files should not be accidentally replaced or mixed up. The public repo should only have code and documentation, not private data, passwords or the local venv folder.

Risk	**What we do about it**	**Related requirements**
Dataset has unexpected columns or values	Check the columns and target before training and fail clearly if something is wrong.	FR-02, SR-05
Personal data ends up in the model	Review the input columns and drop identifiers.	SR-01
Model and encoders come from different runs	Save and share them together and document how to regenerate them.	SR-03, NFR-07
Private data or secrets get committed	Check the repo and .gitignore before pushing.	SR-02
Prediction is treated as certain	Explain the metrics and limitations and call outputs estimates.	NFR-04, FR-13
Bad or unexpected values typed into the web form	Validate every field before preprocessing. preprocess_input also rejects bad values on its own.	FR-17, SR-05
Customer details typed into the app are kept	The app does not save form input to disk or send it anywhere. It only lives in the browser session.	SR-06

3. Detailed Design

3.1 Sequence diagrams

The first diagram shows the normal flow when everything works: the script loads the data, trains and evaluates the model and saves the files.

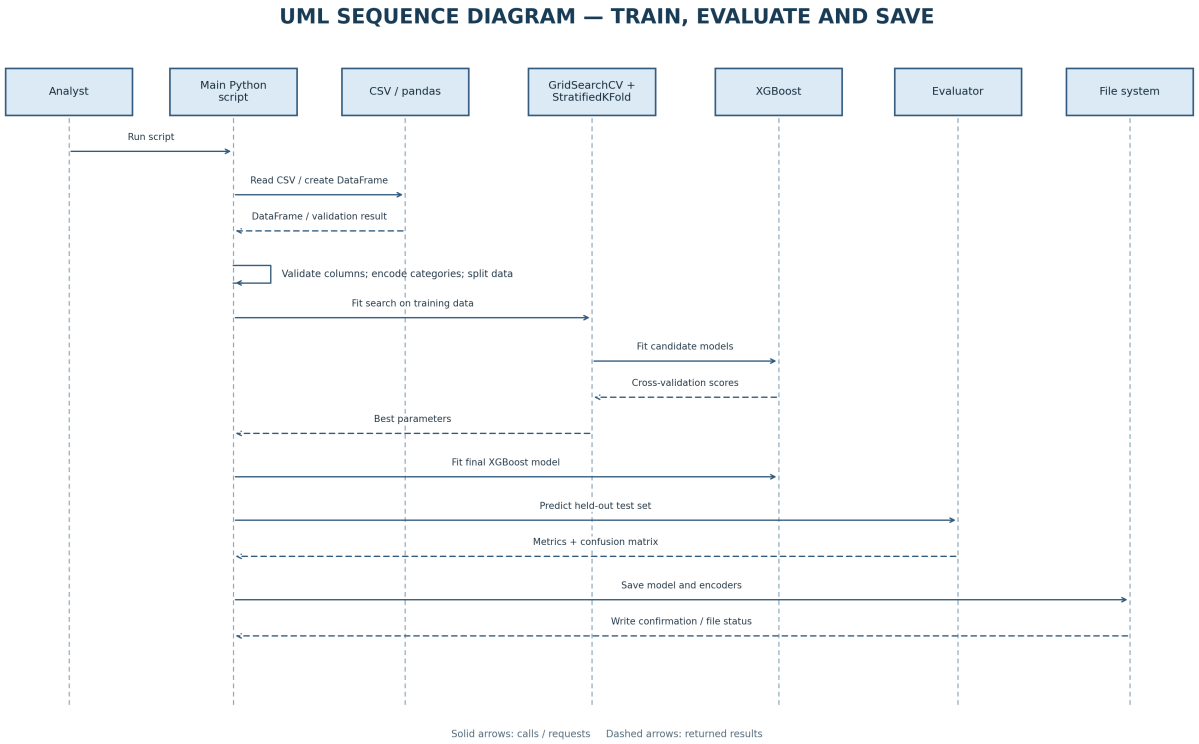


Figure B1. Sequence diagram: train, evaluate and save.

The second diagram shows what happens when the input is bad. If the file or a required column is missing, the program stops before training, prints a readable error and does not claim the run worked. If the input is fine, it moves on to preprocessing, training, evaluation and saving.

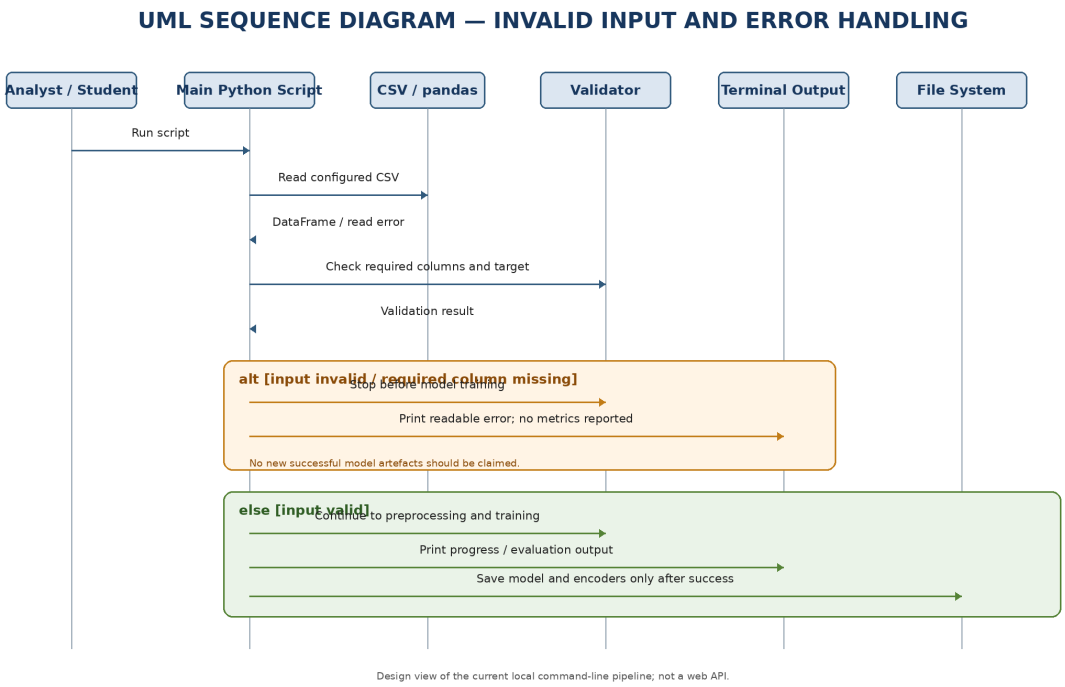


Figure B2. Sequence diagram: invalid input and error handling.

Figure B5 shows the flow for one prediction in the web app. The input is cleaned and validated first. If anything is wrong, every problem is shown and no prediction is made. Otherwise the values are encoded with the saved encoders, the model gives a churn probability, and the app shows the label, risk tier and suggested action.

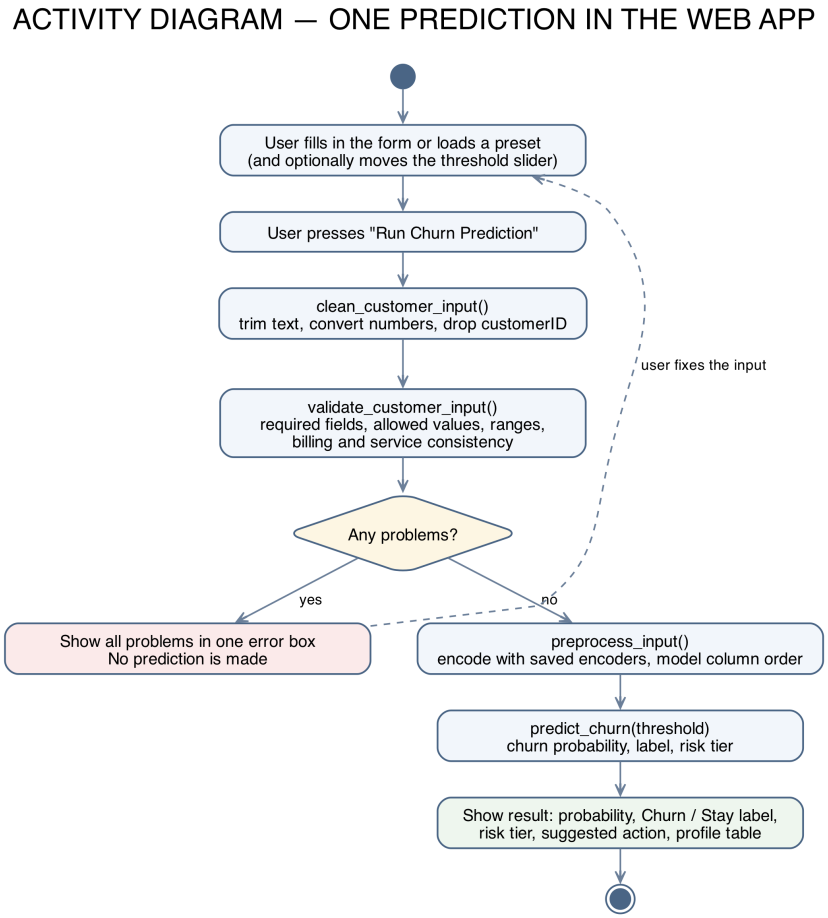


Figure B5. Activity diagram: one prediction in the web app.

3.2 Class design

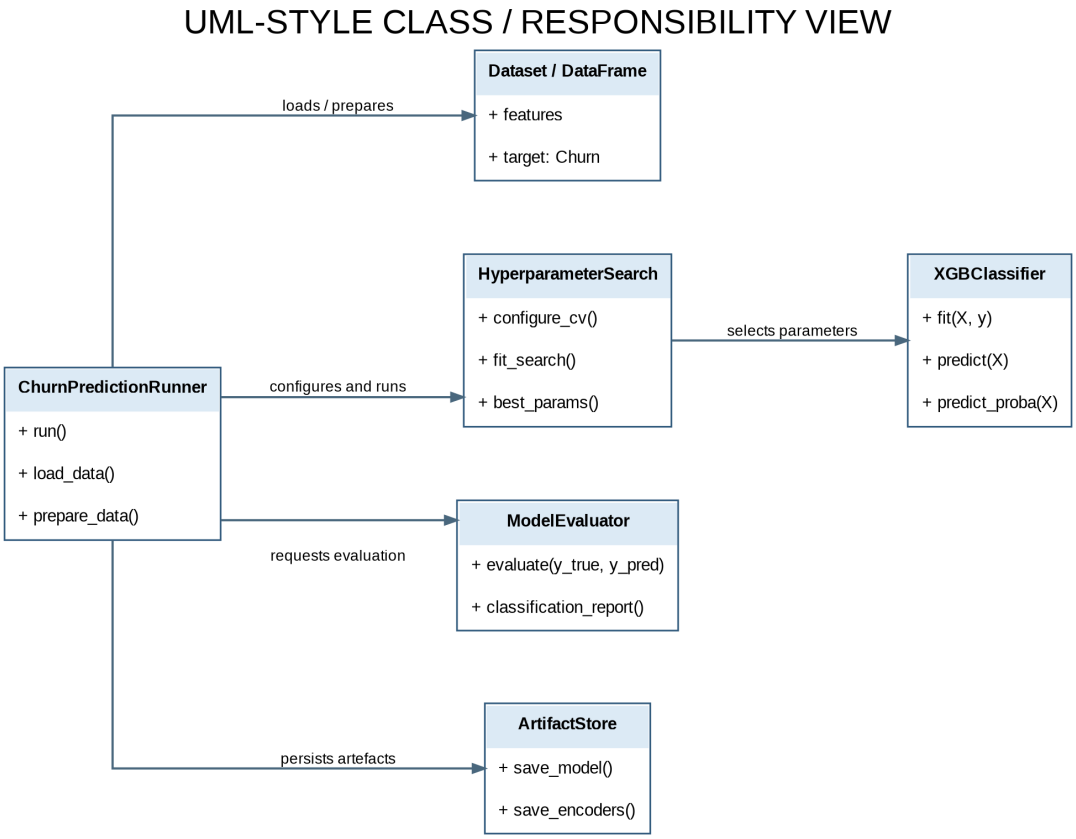


Figure B3. Class diagram showing the main responsibilities and how they connect.

Figure B3 covers the training script. The web app adds two modules: prediction_pipeline.py (functions for loading files, validation, preprocessing, prediction and evaluation) and customer_presets.py (the sample profiles). app.py uses both and has no model logic of its own.

3.3 Model training steps

1. Read the CSV file.
2. Check that the target column and required input columns are there.
3. Prepare the target labels and encode the text columns.
4. Split the data into training and test sets.
5. Set up XGBoost with class weighting.
6. Run GridSearchCV with StratifiedKFold over the parameter grid.
7. Pick the best model based on the search score.
8. Predict on the test set and calculate the metrics.

9. Save the model and encoders together.

3.4 Hyperparameter search

GridSearchCV tried 12 combinations of learning_rate, n_estimators and max_depth (subsample, colsample_bytree and scale_pos_weight were fixed), each on 3 stratified folds. The best setup it found used learning_rate = 0.1, n_estimators = 100 and max_depth = 5. The search only uses the training folds. The test set is used once at the end for the final evaluation.

3.5 Data design

Data object	**Purpose**	**How it is handled**
Raw CSV	Customer records and churn labels.	Read from disk and checked before training.
Feature matrix X	Input columns given to XGBoost.	Text columns encoded, target removed.
Target vector y	Churn / no churn labels.	Used for stratified splitting and training.
Trained model	The chosen XGBoost model.	Saved as customer_churn_model_xgboost.pkl
Encoders	Mappings for the text columns.	Saved as encoders.pkl and always used with the matching model.
Evaluation results	Accuracy, precision, recall, confusion matrix, classification report.	Printed to the terminal by the training script and shown on the web app's evaluation page.
Customer form record	One customer's 19 field values from the web form.	Cleaned and validated, then encoded with the saved encoders. Not saved to disk.
Sample presets	Example customers for demos.	Stored in customer_presets.py. Each one passes validation.
Saved benchmark metrics	The test-set results of the submitted model.	Stored in prediction_pipeline.py and compared with live evaluation results.

3.6 Error handling

Situation	What should happen
Wrong CSV path	Stop and show a file not found error.
Target column missing	Stop before training and say which column is missing.
Unexpected values in the data	Fail clearly or only apply our documented preprocessing. Never make up target labels.
Library not installed	Show the import error and tell the user to install the requirements.
Training or search fails	Show the error and do not say new model files were saved.
Saving files fails	Show the file error and check that the old files are still okay.
Metrics cannot be calculated	Say so clearly instead of printing a misleading number.
A form field is left empty	List it as a missing field and do not predict.
A value is not allowed or out of range	Say which field is wrong and what values are allowed.
Impossible service combination (for example no internet but streaming TV)	Explain which fields conflict and do not predict.
Model or encoder file missing when the app starts	Show an error on the page and do not offer predictions.
Live evaluation does not match the saved numbers	Show a warning instead of a success message.

3.7 Interface design

There are two interfaces. The training script uses the terminal: it shows which model and test split were used, labels each metric clearly, and prints the confusion matrix and classification report. The web app has three pages, picked from the sidebar:

- Customer Churn Prediction: a form with 19 labelled fields in three sections, a preset picker, a Clear form button, an option to estimate Total Charges, and a decision threshold slider. The result shows the churn probability, the churn / stay label, the risk tier, a suggested action, a probability chart and the profile that was scored.
- Model Evaluation: the saved test-set metrics with a confusion matrix chart, and a live check that re-scores the model and compares the result with the saved numbers.
- About the Project: the business goal, the architecture and the limitations.

There is no REST endpoint or downloadable report in this version.

3.8 API design

There is still no REST or other network API. Instead, prediction_pipeline.py gives a small Python interface that the web app and the tests use:

- load_artifacts(): loads the model, the feature order and the encoders.
- clean_customer_input() and validate_customer_input(): tidy the input and return (is_valid, errors).
- preprocess_input() and predict_churn(): encode the record and return the class, churn probability and risk tier at a given threshold.
- evaluate_model_on_dataset() and compare_with_benchmark(): score the held-out test set and compare the result with the saved numbers.

If we add a network API later, it will be designed separately. It would reuse the same validation and prediction functions, return proper error messages, and not send customer data to any outside service.

3.9 Limitations and future work

- The web app only runs locally. Hosting it would need user accounts, HTTPS and a privacy review first.
- The risk tier cut-offs (35% and 65%) are business rules we chose. They could be tuned with real retention cost data.
- Model monitoring and retraining could be added once there is a proper deployment plan.
- Before any real business use, the model should be tested on data from other time periods or customer groups.

Part C: Software Test Plan and Test Cases

This part follows the general layout of IEEE 829 and covers testing of our local Python training pipeline and the Streamlit web app. TC-01 to TC-13 cover the training pipeline and are marked Not Run until we run them by hand and record the results. TC-14 to TC-21 cover the web app and the prediction pipeline. They are automated with pytest, so their status comes from our test suite.

1. Introduction

1.1 Test plan identifier

Field	Value
Identifier	STP-CCP-1.0
Project	Customer Churn Prediction Model
Version	1.1 (updated for the web app)
Related documents	Part A (SRS) and Part B (SAD) of this document
Test environment	Our Python environment and dependencies. OS and versions will be noted when we run the tests.

1.2 Purpose and scope

The aim of this test plan is to check that the pipeline reads the dataset correctly, prepares the features, trains and evaluates the XGBoost model, handles class imbalance properly, reports the right metrics and saves files that can be reused. We also check reliability, repeatability, privacy and what goes into our GitHub repo. Passing these tests does not mean the model will work well for every company or customer group.

For the web app, we check that the form is clear, bad input is rejected with readable messages, the prediction matches what the pipeline gives for the same customer, and the evaluation page shows the right results.

1.3 References

- Part A: Software Requirements Specification (this document)
- Part B: Software Architecture and Design Specification (this document)
- Our project README and requirements file
- Official documentation for Python, XGBoost and scikit-learn

2. Test Items

Test item	What we check
Dataset loading	CSV path, whether it can be read, and the expected columns.
Preprocessing	Target separation, encoding and feature preparation.
Training setup	XGBoost settings, class weighting and the parameter grid.
Cross validation search	GridSearchCV and stratified fold setup.
Evaluation	Predictions, accuracy, precision, recall, classification report and confusion matrix.
Saving files	Model and encoder pickle files are created and can be loaded.
Environment / repo	Dependencies are listed, and the venv folder and private data are not in the repo.
Web app form and presets	All 19 fields have labels and help text; presets fill the form and stay loaded.
Input validation	Missing fields, wrong types, out-of-range numbers and impossible combinations are caught.
Prediction and risk tier	Probability, label at the chosen threshold, risk tier and suggested action.
Evaluation page	Saved and live metrics, confusion matrix chart and the benchmark check.
CI workflow	GitHub Actions installs the requirements and runs pytest on every push and pull request.

3. Features to be Tested

Feature	Requirements	Test cases
Loading the dataset and checking input	FR-01, FR-02, NFR-02	TC-01, TC-02
Target separation and encoding	FR-03, FR-04	TC-03
Training and class imbalance	FR-05, FR-10	TC-04, TC-07
Grid search and stratified CV	FR-06, FR-11, NFR-03	TC-05, TC-13
Metrics output	FR-07, FR-08, FR-12, NFR-04	TC-06
Saving and reloading files	FR-09, FR-14, NFR-07	TC-08, TC-09
Portability and error handling	NFR-02, NFR-06, NFR-10	TC-01, TC-02, TC-10
Privacy and repo hygiene	SR-01 to SR-04, SR-06	TC-11
Documentation	FR-13, NFR-05, NFR-08	TC-12
Web form and presets	FR-15, FR-16, UI-02	TC-14, TC-15
Input validation	FR-17, UI-03, SR-05	TC-16
Prediction, threshold and risk tier	FR-18, FR-19, UI-04, NFR-11	TC-17, TC-18
Evaluation page	FR-20, FR-21, UI-05	TC-19
Missing model files in the app	SW-03	TC-20
Automated tests and CI	NFR-12	TC-21

4. Features Not to be Tested

- Whether every flagged customer will really churn in the future.
- Retention campaigns, CRM connection or a network API, since these are not built.
- The internal correctness of third party libraries (we only check that we use them properly).
- How the model performs for every telecom company or future customer group.
- Formal security certification or legal compliance.
- How the web app looks in every browser or screen size. We test the page logic with Streamlit's AppTest, not the visual layout.

5. Approach

5.1 Security checks

Check	How	Passes if	Test
Data minimisation	Look at the columns used for training.	No personal identifiers are used as inputs.	TC-11
Repo hygiene	Look at the files in the repo and the .gitignore.	No passwords or private data are committed, and the venv folder is not needed.	TC-11
Input handling	Try a wrong path and a CSV without the target column.	The program fails clearly and does not report a successful run.	TC-01, TC-02
File integrity	Load the saved model and its encoder file.	Both load fine and are treated as a pair.	TC-09

5.2 Test levels

- Unit tests: file path, target column check, encoding and metric calculations.
- Integration tests: run the full flow from CSV to model to evaluation.
- System tests: run the main script in our environment and check all outputs.
- Non functional tests: runtime, repeatability, portability and readable output.
- Security and privacy tests: check model inputs, repo contents and file handling.
- UI tests: run the Streamlit pages headless with Streamlit's AppTest and check the form, presets, error messages, results and the evaluation page.
- Continuous integration: GitHub Actions runs pytest -v on every push and pull request to main.

5.3 Model evaluation steps

1. Note down the dataset file and the installed package versions.
2. Confirm the target label and that churn is the positive class.
3. Use the same train/test split and cross validation setup.
4. Make sure GridSearchCV only uses the training folds and never the test set.
5. Calculate accuracy, precision, recall and the confusion matrix from the test predictions.

6. Look at recall and precision along with accuracy, since false negatives and false positives mean different things.
7. Record the final settings and the actual results from the run.

5.4 Test data and environment

We will use the project dataset for the full end to end test, and small dummy CSV files to test bad input. For each run we will note the OS, Python version, library versions, CPU and RAM, how long it took and the exact command used. We will not use any real customer data other than the project dataset.

6. Item Pass/Fail Criteria

A test passes if what actually happens matches what we expected. A test fails if the program gives wrong output, crashes without a clear error, uses the wrong target or features, does not save the files, or gives a result we cannot reproduce.

Severity	**Meaning**
Critical	Private data is exposed, the target leaks into the inputs, or the test set was used for tuning.
Major	A required step, metric or file saving step does not work.
Minor	A documentation, label or formatting issue that does not change how the model behaves.

7. Entry, Exit, Suspension and Resumption Criteria

Entry criteria	**Exit criteria**
The dataset and script are ready, dependencies are installed, and we know the target column and output paths.	All Must tests pass or have their defects written down, the actual metrics and settings are recorded, and the output files have been checked.

We will pause testing if the dataset is missing, the script will not run in our environment, or we find a privacy or data leakage problem. Once it is fixed, we will rerun the affected tests and do one full run to make sure everything still works.

8. Test Deliverables

- Filled in test execution table with pass/fail status and proof.

- Terminal output showing the actual metrics.
- The hyperparameter settings and cross validation setup used.
- Proof that the model and encoder files were created and loaded.
- Notes on any bugs found and how we fixed them.
- The pytest report and the GitHub Actions run for the automated tests (TC-14 to TC-21).
- Updated SRS / SAD if the program ends up behaving differently from this document.

9. Responsibilities and Schedule

Role	**Responsibility**
Akshaya B: ML pipeline and integration lead	Owns the training script, the saved model and encoder files, the CI workflow and the threshold / risk tier feature. Runs TC-01 to TC-13, records the results and fixes pipeline bugs.
Ashwin C H: Web app and automated test developer	Builds the Streamlit form, presets, input validation and evaluation page, and writes the pytest suite for TC-14 to TC-21. Fixes bugs found in the app.
Amar Patil: Test reviewer and documentation lead	Goes through the test results, checks that every requirement is covered in the traceability matrix, runs the repo and documentation checks (TC-11, TC-12) and keeps the SRS / SAD up to date.
Instructor	Review the submitted documents and the demo.

Order we will follow: input checks first, then preprocessing, training and evaluation, then file checks, portability checks and finally a documentation review.

10. Risks and Contingencies

Risk	Impact	What we will do
Dataset path or columns change	Training fails or features get mixed up.	Check column names and target before running, and update the docs if they change.
Class imbalance hides poor churn detection	Accuracy looks fine even though recall is low.	Always report precision, recall and the confusion matrix with accuracy.
Overfitting	Training scores look better than real performance.	Compare training and test scores and always keep a separate test set.
Library version mismatch	Saved pickle files may not load.	Note the versions and regenerate the model and encoders together if needed.
Output files missing or too large for the repo	Others may not be able to reproduce our results.	Mention which files are in the repo and give clear setup and run steps.
Report numbers differ from the latest run	The documentation becomes wrong.	Always take the final numbers from the actual terminal output.

11. Test Cases

11.1 Test case list

We have 21 test cases covering the functional, non functional and security requirements. TC-01 to TC-13 are marked Not Run until we run them by hand. TC-14 to TC-21 are automated, and their status comes from our pytest suite (94 tests, all passing).

Test ID	Test name	Req. IDs	Steps	Expected result	Status
TC-01	Load valid CSV	FR-01	Run the script with the correct CSV in place.	File is read and the program moves on to preprocessing.	Not Run
TC-02	Missing target / bad input	FR-02, NFR-02, SR-05	Use a dummy CSV with no target column, or point to a file that does not exist.	Script stops with a clear error and no metrics are printed.	Not Run

TC-03	Target separation and encoding	FR-03, FR-04	Check the prepared features and target, or use a small test CSV.	Target is separate from the inputs and text columns are turned into numbers.	Not Run
TC-04	Model training	FR-05, FR-10	Run the normal training pipeline.	XGBoost training finishes and class weighting is applied.	Not Run
TC-05	Grid search setup	FR-06, FR-11, NFR-03	Run GridSearchCV and check its output.	Stratified CV is used, the best settings are printed, and 12 combinations x 3 folds (36 fits) are run.	Not Run
TC-06	Evaluation metrics	FR-07, FR-08, FR-12, NFR-04	Do a full run and check the report and confusion matrix.	Accuracy, precision, recall and confusion matrix are shown for the test data.	Not Run
TC-07	Class imbalance handling	FR-10	Check the XGBoost settings and training output.	Class weighting is set, and the test set is not downsampled.	Not Run
TC-08	Save files	FR-09	Run the pipeline fully and look at the output folder.	Model and encoder files exist at the expected paths.	Not Run
TC-09	Reload saved files	FR-14, NFR-07, SR-03	Load the saved model and encoders in Python.	Both load without errors and are from the same run.	Not Run
TC-10	Fresh environment	NFR-06, NFR-10	Follow the README setup in a new environment.	Dependencies install and the script runs without our old venv folder.	Not Run

TC-11	Privacy and repo check	NFR-09, SR-01, SR-02, SR-04, SR-06	Check the input columns, repo files and .gitignore, and that the web app does not write form input to disk.	No personal identifiers used, no secrets or private data committed, nothing sent outside.	Not Run
TC-12	Documentation check	FR-13, NFR-05, NFR-08	Ask a classmate to follow the README and read the output.	They can find the setup steps, model, metrics, file paths and limitations without help.	Not Run
TC-13	Repeatability	NFR-03	Run twice with the same data, settings, seed and versions.	Split and results are the same (any small difference is noted).	Not Run
TC-14	Form fields and labels	FR-15, UI-02	Start the app and look at the prediction form.	All 19 fields are shown with a label and help text, filled with the default preset.	Pass (automated)
TC-15	Sample presets	FR-16	Load each preset, submit, switch pages and come back; press Clear form.	Each preset is valid, high / low risk presets get the matching prediction, and loaded values stay after submitting or switching pages.	Pass (automated)
TC-16	Input validation	FR-17, UI-03, SR-05	Submit an empty form, wrong types, out-of-range numbers and impossible combinations.	Every problem is listed in one error box and no prediction is shown. All 7,043 real dataset rows still pass.	Pass (automated)

TC-17	Prediction result	FR-18, UI-04, NFR-11	Submit a valid customer.	Probability between 0 and 1, Stay + Churn add up to 1, label and notice shown within about 2 seconds.	Pass (automated)
TC-18	Threshold and risk tier	FR-19	Predict at different thresholds and at probabilities near 35% and 65%.	The label follows the threshold and the risk tier is Low / Moderate / High at the right cut-offs.	Pass (automated)
TC-19	Evaluation page	FR-20, FR-21, UI-05	Open Model Evaluation and run the live check at 0.50 and at 0.30.	All metrics and the confusion matrix are shown; at 0.50 the live results match the saved benchmark.	Pass (automated)
TC-20	Missing model files	SW-03	Load the files from a path that does not exist.	A clear file-not-found error is raised instead of a crash.	Pass (automated)
TC-21	CI workflow	NFR-12	Push to a branch, open a pull request to main and check GitHub Actions.	The Customer Churn CI Pipeline workflow installs the requirements on Python 3.11 and pytest -v passes.	Pass (automated)

11.2 Test execution record

We will fill this in after running each test and attach terminal screenshots or logs where needed.

Test ID	**Actual result / proof**	**Pass / Fail**	**Bug / notes**
TC-01		Not Run	
TC-02		Not Run	

TC-03		Not Run	
TC-04		Not Run	
TC-05		Not Run	
TC-06		Not Run	
TC-07		Not Run	
TC-08		Not Run	
TC-09		Not Run	
TC-10		Not Run	
TC-11		Not Run	
TC-12		Not Run	
TC-13		Not Run	
TC-14	test_every_input_has_label_and_help and test_app_starts_with_default_preset pass.	Pass	test_app.py (Streamlit AppTest)
TC-15	Preset tests pass for all 4 presets, plus the submit and page-switch regression tests.	Pass	Fixed a bug where the preset reset to low risk on submit
TC-16	Validation tests pass, including the empty form (19 problems listed) and all dataset rows.	Pass	test_pipeline.py and test_app.py
TC-17	test_successful_prediction_output passes; high-risk preset predicted at about 97%.	Pass	
TC-18	test_predict_churn_custom_threshold, test_threshold_boundary_cases and test_get_risk_tier_classification pass.	Pass	
TC-19	Live evaluation tests pass: 0.50 matches the benchmark, 0.30 shows the custom threshold note.	Pass	

TC-20	test_missing_model_file_handling and test_missing_encoder_file_handling pass.	Pass	
TC-21	Recent GitHub Actions runs on main and on pull request branches passed.	Pass	Python 3.11, pytest -v

12. Traceability Matrix (SRS to Test Cases)

Requirement	Test case(s)
FR-01	TC-01
FR-02	TC-01, TC-02
FR-03	TC-03
FR-04	TC-03
FR-05	TC-04
FR-06	TC-05
FR-07	TC-06
FR-08	TC-06
FR-09	TC-08
FR-10	TC-04, TC-07
FR-11	TC-05
FR-12	TC-06
FR-13	TC-12
FR-14	TC-09
FR-15	TC-14
FR-16	TC-15
FR-17	TC-16
FR-18	TC-17
FR-19	TC-18
FR-20	TC-19
FR-21	TC-19
NFR-01	Runtime noted during TC-04 / TC-10
NFR-02	TC-01, TC-02
NFR-03	TC-05, TC-13
NFR-04	TC-06, TC-07
NFR-05	TC-12
NFR-06	TC-10
NFR-07	TC-09
NFR-08	TC-06, TC-12
NFR-09	TC-11
NFR-10	TC-01, TC-10

NFR-11	TC-17
NFR-12	TC-21
SR-01	TC-11
SR-02	TC-11
SR-03	TC-09
SR-04	TC-11
SR-05	TC-02, TC-16
SR-06	TC-11
UI-01	TC-06
UI-02	TC-14
UI-03	TC-16
UI-04	TC-17
UI-05	TC-19
SW-01	TC-01
SW-02	TC-08
SW-03	TC-20
HW-01	TC-10
COM-01	TC-11

Appendix A: Experiment Record Template

This records the training run that produced the model files we submitted (customer_churn_model_xgboost.pkl and encoders.pkl). The results were checked again on 12 October 2026 by loading those files and scoring the held-out test set.

Field	**Record**
Date / run ID	Model trained on 11 October 2026 and uploaded to our GitHub repository (commit b174068). Re-checked on 12 October 2026 by loading the saved model and scoring the test set again.
Dataset file / version	WA_Fn-UseC_-Telco-Customer-Churn.csv (Telco Customer Churn, 7,043 customers, about 26.5% churn)

Python and library versions	Re-check environment: Python 3.12.7, XGBoost 3.4.1, scikit-learn 1.9.1, pandas 3.0.6, NumPy 2.5.4 (tests also pass on Python 3.11). The saved model was made with an older XGBoost release (XGBoost shows a version warning when loading it); the exact versions on the original training machine were not written down.
Machine / OS	Re-check machine: Apple M2, macOS 26.3.1 (arm64). The original training machine was not recorded.
Train/test split and random seed	80/20 split, stratified on Churn, random_state = 42: 5,634 training rows (1,495 churn) and 1,409 test rows (374 churn). The same seed (42) is used for XGBoost and for the StratifiedKFold shuffle.
GridSearchCV parameter grid / number of folds	12 combinations: n_estimators (100, 200) x max_depth (3, 4, 5) x learning_rate (0.05, 0.1), with subsample = 0.8, colsample_bytree = 0.8 and scale_pos_weight = 2.77 fixed. StratifiedKFold (folds: 3), so 36 fits in total; scoring = accuracy.
Best hyperparameters	learning_rate = 0.1, n_estimators = 100, max_depth = 5
Class weighting setting	Class weighting enabled in XGBoost to handle the smaller churn class
Training accuracy	About 83% (train/test gap brought down from about 19 points to about 7)
Test accuracy	About 76%
Churn precision	About 53%
Churn recall	About 75%
Confusion matrix	Test set, 1,409 customers (rows = actual, columns = predicted): actual Stay: 785 predicted Stay, 250 predicted Churn; actual Churn: 93 predicted Stay, 281 predicted Churn. So TN = 785, FP = 250, FN = 93, TP = 281.
Runtime	About 5.5 seconds wall-clock for the whole script (loading, 36-fit grid search, evaluation and saving) when re-run on the machine above. The time of the original run was not recorded.
Model file path	customer_churn_model_xgboost.pkl
Encoder file path	encoders.pkl

Notes / limitations	Recall was given priority over precision, so the model catches most churners but also raises more false alarms (about 1 in 2 flagged customers actually churn). Results are from this dataset only. Re-running the training script with a newer XGBoost (3.4.1) picks n_estimators = 200 and gives slightly different numbers (test accuracy 76.1%, confusion matrix 800 / 235 / 102 / 272), so the saved model files are kept as the reference.